

Primes, GCD & Modular Arithmetic

A comprehensive technical reference covering Number Theory, Cryptographic Foundations, and Algorithm Design — from classical Euclidean methods to modern RSA and beyond.

NUMBER THEORY

CRYPTOGRAPHY

ALGORITHM DESIGN

Developed by Talenciaglobal

Topic Classification

Domain Overview

Category: Number Theory / Cryptography / Algorithm Design

Domain: Mathematics → Discrete Mathematics → Computational Number Theory

Difficulty: Intermediate (foundations) to Advanced (cryptographic applications)

Industry Relevance: Critical for TLS, RSA, ECC, hash tables, RNGs, checksums, and competitive programming

Recommended Learning Path

01

Basic Modular Arithmetic

02

Prime Testing

03

GCD & Euclid

04

Modular Inverses

05

RSA & Advanced Crypto



Prerequisites: Basic algebra, loops & recursion, Big-O notation fundamentals.

What Is It? — Formal Definitions

Prime Numbers

Integers greater than 1 with exactly two distinct positive divisors: 1 and itself. Primes are the **multiplicative atoms** of the integers — every integer factors uniquely into primes (Fundamental Theorem of Arithmetic).

GCD — Greatest Common Divisor

The largest positive integer that divides two or more integers without remainder. Example: $\text{GCD}(12, 18) = 6$. Computed efficiently via the Euclidean algorithm in $O(\log n)$ time.

Modular Arithmetic

Arithmetic on congruence classes modulo a positive integer m . Numbers wrap around after reaching m — analogous to "clock arithmetic": $10 + 3 \bmod 12 = 1$.

Key Terminology

Composite number · Prime factorization · Coprime (relatively prime) · Euclidean algorithm · Extended Euclidean algorithm · Modulo operation · Congruence · Residue · Modular inverse · Totient function $\varphi(n)$ · Fermat's Little Theorem · Euler's Theorem

Core Components at a Glance

Component	Primes	GCD	Modular Arithmetic
Core Element	Prime numbers	Euclidean algorithm	Modulus m
Key Operation	Primality testing	$\gcd(a, b)$	$(a + b) \bmod m$, $(a \times b) \bmod m$
Advanced Tool	Sieve of Eratosthenes	Extended Euclid	Modular exponentiation
Crypto Role	RSA modulus generation	Key derivation, inverse	Finite fields, ciphertext



Industry statistic: Over **95% of HTTPS connections** rely on RSA or ECC — both built entirely on these three pillars.



Why Does It Exist? — Problems Solved

Without Primes

No secure asymmetric cryptography. RSA and Diffie-Hellman would collapse — factoring large composites would be trivial, exposing every encrypted channel.

Without Efficient GCD

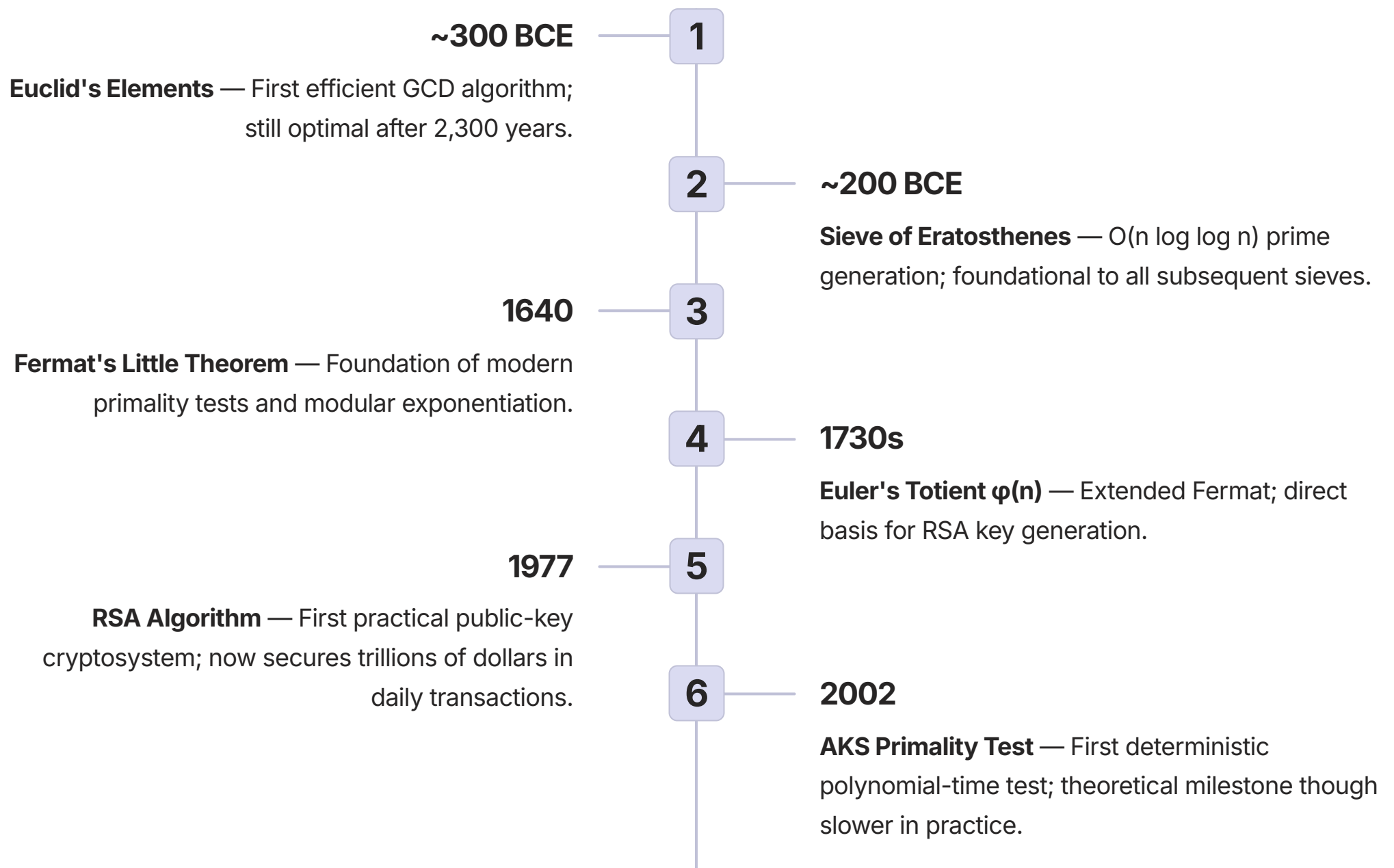
Simplifying fractions becomes computationally expensive. Solving linear Diophantine equations — foundational to key generation — would be impractical at scale.

Without Modular Arithmetic

Computers could not handle the astronomically large integers required in cryptography (e.g., 2048-bit numbers). Hash tables, checksums, and cyclic structures would lose their mathematical foundation.

Factoring a 2048-bit RSA modulus — the product of two large primes — would require **billions of years** on classical hardware. This computational asymmetry is the bedrock of modern internet security.

Evolution & History



Current state: Probabilistic tests (Miller–Rabin, Baillie–PSW) dominate practice. ECC is replacing RSA in many contexts. Post-quantum cryptography (lattice-based) may eventually supersede prime-based schemes.

How It Works — Three Pillars

Sieve of Eratosthenes

Generate all primes up to n in $O(n \log \log n)$:

1. List numbers 2 through n
2. Mark 2 as prime; cross out all multiples of 2
3. Next unmarked number (3) is prime; cross out its multiples
4. Repeat until $\sqrt{n}$ is reached
5. All remaining unmarked numbers are prime

✓ Result for $n=30$: 2, 3, 5, 7, 11, 13, 17, 19, 23, 29

Euclidean GCD

$\text{gcd}(a, b) = \text{gcd}(b, a \bmod b)$, until $b=0$

$\text{gcd}(270, 192)$:

$270 \bmod 192 = 78 \rightarrow \text{gcd}(192, 78)$

$192 \bmod 78 = 36 \rightarrow \text{gcd}(78, 36)$

$78 \bmod 36 = 6 \rightarrow \text{gcd}(36, 6)$

$36 \bmod 6 = 0 \rightarrow \text{return } 6$

Modular Exponentiation

Compute $a^b \bmod m$ in $O(\log b)$ via repeated squaring:

```
function mod_pow(base, exp, mod):  
    result = 1  
    base = base % mod  
    while exp > 0:  
        if exp is odd:  
            result = (result * base) % mod  
        base = (base * base) % mod  
        exp //= 2  
    return result
```

Example: $3^{13} \bmod 10 = 3$

Miller–Rabin (One Round)

Input: n (odd > 2)

Write $n-1 = 2^s * d$ (d odd)

Pick random a in $[2, n-2]$

$x = a^d \bmod n$

if $x == 1$ or $x == n-1 \rightarrow$ probably prime

for $r = 1$ to $s-1$:

$x = x^2 \bmod n$

if $x == n-1 \rightarrow$ probably prime

if none $\rightarrow$ composite

Types & Variants

Prime Number Variants

Type	Definition	Use Case
Probable Prime	Passes a probabilistic test (e.g., Miller–Rabin)	RSA key generation
Strong Prime	$p-1$ and $p+1$ have large prime factors	More secure RSA (avoiding certain attacks)
Safe Prime	$p = 2q+1$ with q prime	Diffie-Hellman groups
Mersenne Prime	$2^p - 1$ where p is prime	Large primes for GIMPS, RNGs

GCD Algorithm Variants

Variant	Complexity	Best For
Euclidean (subtraction)	$O(n)$ worst-case	Historical only
Euclidean (modulo)	$O(\log \min(a,b))$	General purpose
Binary GCD (Stein)	Shift-based	Embedded hardware
Extended Euclidean	$O(\log \min(a,b))$	Modular inverses

Modular Arithmetic Systems

System	Modulus	Characteristics
$\mathbb{Z}/m\mathbb{Z}$	Composite or prime	Ring; not a field if composite
$\text{GF}(p)$	Prime p	Field — every non-zero has inverse
$\text{GF}(2^n)$	Irreducible polynomial	AES, ECC binary fields

Real-World Use Cases



RSA Key Generation — Web Servers

Industry: Cloud / Web Hosting.

Generate a 2048-bit RSA key pair for HTTPS. Miller–Rabin (64 rounds) tests primality; extended Euclidean computes private key $d = e^{-1} \bmod \varphi$. OpenSSL completes keygen in under 1 second on modern CPUs. **Stat:** Over 1.8 billion websites use TLS — every one depends on this process.



Consistent Hashing — CDN Caching

Industry: Content Delivery Networks.

Map keys to cache servers on a circle modulo 2^{32} . When servers change, only **K/N keys** are remapped (vs. nearly all with naive modulo hashing). Virtual nodes use modulo to spread load evenly across physical servers.



ISBN-13 / Luhn Checksum — Banking

Industry: E-commerce & Banking. Luhn algorithm doubles every second digit, sums all digits; total **modulo 10 must equal 0**. Provides real-time validation without database lookup. Mod 10 catches all single-digit errors; mod 11 (ISBN-10) catches transpositions.

Implementations & Code Examples

Example 1: Sieve of Eratosthenes (Python)

```
def sieve_of_eratosthenes(n: int) -> list[int]:
    """Return list of all primes <= n using O(n log log n) sieve."""
    if n < 2:
        return []
    is_prime = [True] * (n + 1)
    is_prime[0] = is_prime[1] = False
    p = 2
    while p * p <= n:
        if is_prime[p]:
            for multiple in range(p * p, n + 1, p):
                is_prime[multiple] = False
            p += 1
    return [i for i, prime in enumerate(is_prime) if prime]

# Example
print(sieve_of_eratosthenes(30)) # [2,3,5,7,11,13,17,19,23,29]
```

Example 2: Extended Euclidean (Modular Inverse)

```
def egcd(a: int, b: int) -> tuple[int, int, int]:
    """Return (g, x, y) s.t. a*x + b*y = g = gcd(a,b)."""
    if b == 0:
        return (a, 1, 0)
    g, x1, y1 = egcd(b, a % b)
    return (g, y1, x1 - (a // b) * y1)

def mod_inverse(a: int, m: int) -> int:
    """Return x s.t. a*x ≡ 1 mod m. Requires gcd(a,m)=1."""
    g, x, _ = egcd(a, m)
    if g != 1:
        raise ValueError(f"Inverse doesn't exist, gcd={g}")
    return x % m

# inverse of 3 mod 11 → 4 (since 3*4=12≡1 mod 11)
print(mod_inverse(3, 11)) # 4
```

Example 3: Modular Exponentiation

```
def mod_pow(base: int, exp: int, mod: int) -> int:
    """Compute (base^exp) % mod in O(log exp) time."""
    result = 1
    base %= mod
    while exp > 0:
        if exp & 1:
            result = (result * base) % mod
        base = (base * base) % mod
        exp >>= 1
    return result

# 2^10 mod 1000 = 24
print(mod_pow(2, 10, 1000)) # 24

# RSA-like: 65^17 mod 3233
# (n=3233=61*53, textbook example)
print(mod_pow(65, 17, 3233)) # 2790
```

Hands-On Labs

Lab 1: Miller–Rabin Primality Tester

Objective: Implement a probabilistic primality test and compare with trial division.

Steps:

1. Write `miller_rabin(n, k=40)` performing k rounds
2. Write `trial_division(n)` checking divisors up to $\sqrt{n}$
3. Test on prime 1000003 and composite 1000003×1000037
4. Measure time for both on $n \sim 1e12$

Validation: For $n=1000003$, trial division $\sim 1\text{ms}$; for the product, trial division ~ 1 second, Miller–Rabin $< 1\text{ms}$ in both cases.

✓ Learning outcome: Probabilistic tests are vastly faster; error probability can be made negligible with sufficient rounds.

Lab 2: RSA from Scratch

ADVANCED

Objective: Generate a small RSA key pair, encrypt a message, decrypt, and break it by factoring n .

Steps:

1. Generate two 15-bit primes $p=32117$, $q=32119$
2. Compute $n=p \times q$, $\phi=(p-1)(q-1)$; choose $e=65537$; verify $\gcd(e, \phi)=1$
3. Compute $d = e^{-1} \bmod \phi$ via extended Euclidean
4. Encrypt "Hi" $\rightarrow m=18537$; compute $c = m^e \bmod n$
5. Decrypt: $c^d \bmod n \rightarrow$ recover m
6. Break: factor n by trial division up to $\sqrt{n}$; recover p , q , then d

⚠ Learning outcome: Understand why p , q must be hundreds of bits for real-world security. Factorization of 30-bit numbers takes < 1 second; 2048-bit takes billions of years.

Performance Considerations

Algorithm Complexity Reference

Algorithm	Time Complexity	Space Complexity
Sieve of Eratosthenes (up to n)	$O(n \log \log n)$	$O(n)$
Trial division primality test	$O(\sqrt{n})$	$O(1)$
Miller–Rabin (k rounds, n-bit)	$O(k \log^3 n)$	$O(\log n)$
Euclidean GCD	$O(\log \min(a,b))$	$O(1)$
Modular exponentiation (b-bit exp)	$O(\log b)$ multiplications	$O(1)$
Extended Euclidean	$O(\log \min(a,b))$	$O(1)$

<0.5ms

Miller–Rabin

On a 2048-bit candidate on modern CPU hardware

~0.5s

Sieve of 10^8

Optimized C implementation; ~20s in pure Python

$O(\ln n)$

Expected Candidates

Average candidates tested per prime found during RSA keygen

100k

TLS Connections/s

Achievable with dedicated crypto cores using $O(\log n)$ per connection



Common bottlenecks: Repeated modular multiplication without exponentiation; trial division on large numbers; recursive GCD stack overflow. Use iterative GCD in production.

Scalability & Reliability

Scale	Prime Generation	GCD	Modular Exponentiation
Small ($n < 10^6$)	Sieve, trial division	Euclid	Direct exponentiation
Medium ($10^6 - 10^{12}$)	Miller–Rabin + small trial division	Euclid	Binary exponentiation
Large ($n \sim 2^{2048}$)	CSPRNG + Miller–Rabin (k=64)	Euclid (big-int built-in)	Montgomery + sliding window
Massive (distributed)	GIMPS-style Mersenne search	Not needed (single node)	Multi-precision libraries (GMP)

High Availability

For services performing RSA signing at scale, use **Hardware Security Modules (HSMs)** to offload modular exponentiation. Protect against timing attacks with blinding. Stateless by nature — except for sieve which precomputes primes once.

Optimization Techniques

- **Precomputation:** Small prime list for fast composite filtering
- **Montgomery reduction:** Faster modular multiplication
- **Constant-time algorithms:** Avoid timing side-channels in crypto
- **Early abort:** Check small divisors before Miller–Rabin

Security Considerations

Threat Model & Common Vulnerabilities

Vulnerability	Cause	Mitigation
Small $p-1$ factor (Pohlig-Hellman)	$p-1$ is smooth	Use safe primes or test $p-1$ factors
Common modulus attack	Two parties share n	Always generate a fresh modulus per deployment
Blinding bypass	No randomization in exponentiation	Add random masking before exponentiation
Side-channel (cache timing)	Secret-dependent memory access	Use constant-time code (Montgomery ladder)
Carmichael number bypass	Over-reliance on Fermat test	Use Miller–Rabin (Carmichael fails with high probability)

1

Use Vetted Libraries

Always use OpenSSL, libsodium, or PyCryptodome. Never hand-roll RSA or ECC for production systems.

2

Choose $e=65537$

It is prime, a Fermat number ($2^{16}+1$), and has only two 1-bits in binary — enabling fast encryption with minimal security trade-offs.

3

Validate All Inputs

Verify modulus is non-zero, $\gcd(e, \phi)=1$, and regenerate primes if $\gcd(p-1, e) \neq 1$. Compliance: FIPS 186-5 for RSA key generation.

Best Practices — Do's & Don'ts

✓ Do's

Area	Recommendation
Design	Use built-in arbitrary precision integers (Python int, Java BigInteger)
Development	Write unit tests with known answers from published test vectors
Security	Use Miller–Rabin with at least 40 rounds for 2048-bit primes
Performance	Precompute small primes list for trial division up to 10^5
Operations	Rotate cryptographic keys regularly; use HSMs for private keys

× Don'ts

Area	Avoid
Design	Don't use Fermat primality test alone — Carmichael numbers pass
Development	Don't implement RSA or ECC from scratch for production
Security	Don't leak timing information (e.g., early exit in GCD if $a=b$)
Performance	Don't compute modular inverse via brute force — use extended Euclid
Operations	Don't hardcode primes — generate fresh per deployment

Common Mistakes & Pitfalls



Beginner Pitfalls

- **% vs. true modulus:** Use `((a % m) + m) % m` for negative inputs
- **Trial division to n:** Loop only while $i*i \leq n$ — $O(\sqrt{n})$ not $O(n)$
- **1 is not prime:** Start sieve from 2; mark index 0 and 1 as False



Intermediate Pitfalls

- **pow(base, exp) then % mod:** Causes huge intermediate numbers — use `pow(base, exp, mod)`
- **gcd(e, φ) ≠ 1:** Verify before proceeding; regenerate primes if needed
- **Too few Miller–Rabin rounds:** 1 round gives >25% error — use ≥ 40 for crypto



Production / Advanced Pitfalls

- **Timing side-channel:** Use Montgomery ladder or blinding for secret exponents
- **Shared prime p across RSA keys:** GCD attack breaks both keys instantly
- **Naïve loop for $a^b \bmod m$:** For $b=2^{106}$ this is impossible — always use binary exponentiation

- ⊗ Production incident: In 2012, researchers scanned 7.1 million RSA public keys and found that **0.2% shared a prime factor** — allowing immediate private key recovery via GCD. Always generate fresh, independent primes.

Comparison with Alternatives

Primality Testing Methods

Method	Deterministic	Time (n-bit)	Practical Use Case
Trial Division	Yes	$O(2^{(n/2)})$	$n < 10^{12}$
Fermat (base 2)	No	$O(n^2)$	None — unsafe (Carmichael numbers)
Miller–Rabin	No (probabilistic)	$O(k\ n^3)$	RSA keygen, all crypto
AKS	Yes	$O(n^6)$ slow	Theoretical; not practical
Baillie–PSW	Heuristic	$O(n^3)$	General purpose; no known counterexample

Modular Arithmetic Libraries

Library	Speed	Constant-Time	Language
GMP	Very high	No	C
OpenSSL BN	High	Yes (blinding)	C
Python int	Moderate	High (big-int in C)	Python
libsodium	High	Very high (timing-safe)	C



Decision rule: Production crypto → use OpenSSL / libsodium / PyCryptodome. Performance-critical non-crypto → GMP. Learning / prototyping → Python built-in integers with own Miller–Rabin.

Advantages, Disadvantages & Trade-offs

Primes Advantages: Uniqueness of factorization; infinite supply; foundation of all asymmetric cryptography. Disadvantages: Generating large random primes is slow; no simple closed-form formula for the n-th prime. Enterprise note: Must use known-safe libraries (OpenSSL); never hand-rolled.	GCD Advantages: Extremely fast $O(\log n)$; simple to implement correctly; one of the oldest optimal algorithms known. Disadvantages: None significant — the algorithm is provably optimal. Enterprise note: Safe to hand-roll; Euclid is simple and well-understood.	Modular Arithmetic Advantages: Bounds numbers to fixed size; enables finite fields; supports constant-time operations. Disadvantages: Loss of natural ordering; division requires modular inverse (which may not exist for composite moduli). Enterprise note: Use libraries for crypto; hand-rolling is acceptable for checksums and hashing.
---	--	--

Speed vs. Security Trade-off (Primality Testing)

Approach	Speed	Security	Complexity
Deterministic AKS	Very slow	Perfect	High
Probabilistic Miller–Rabin (k rounds)	Fast	Adjustable (2^{-k} error)	Low
Trial division	Impractical for large n	Perfect	Very low

Design & Architectural Considerations

Key Architecture Decisions

→ Primality Test Selection

Deterministic for $n < 2^{64}$ using known bases.

Probabilistic (Miller–Rabin, error $< 2^{-128}$) for larger.

Never use Fermat test alone — Carmichael numbers are a known attack vector.

→ Modular Representation

Native integers (Python int, Java BigInteger) for most use cases. GMP or OpenSSL bignum for extreme performance. Fixed-width uint64 with carry handling for embedded systems.

→ GCD Trade-off

Binary GCD (Stein) on microcontrollers lacking division hardware. Standard Euclidean algorithm everywhere else — simpler, equally fast on modern CPUs.

Design Patterns

- **Template Method:** Prime sieve with different marking strategies (bitset, segmented sieve)
- **Strategy:** Pluggable modular multiplication algorithms (Montgomery, Barrett)
- **Singleton:** Precomputed small prime table shared across the application

Maintainability & Observability

- Implement number theory as pure functions; avoid global state
- Log prime generation rounds for diagnostics
- Measure modular exponentiation time to detect side-channel leaks
- Use arbitrary-precision integers (Python int) for high reliability at moderate speed

Learning Summary & Revision Cheat Sheet

Core Takeaways

- Primes are multiplicative atoms; primality testing is fast (probabilistic) but factoring is slow — the basis of RSA
- Euclidean GCD runs in $O(\log n)$ — one of the oldest and still optimal algorithms in computer science
- Modular exponentiation uses repeated squaring for $O(\log b)$ efficiency
- Extended Euclidean yields modular inverses — essential for RSA private key and crypto protocols
- Never roll your own crypto arithmetic for production; use constant-time, blinding, and well-tested libraries

Key Theorems

Fermat's Little Theorem: If p is prime and a is not a multiple of p :

$$a^{p-1} \equiv 1 \pmod{p}$$

Euler's Theorem: If $\gcd(a, n) = 1$:

$$a^{\varphi(n)} \equiv 1 \pmod{n}$$

RSA Key Generation:

$$n = p \times q, \quad \varphi(n) = (p-1)(q-1), \quad e \cdot d \equiv 1 \pmod{\varphi(n)}$$

Algorithm Complexity Quick Reference

Operation	Algorithm	Complexity
List primes up to N	Sieve of Eratosthenes	$O(N \log \log N)$
Test single prime (large)	Miller–Rabin (k rounds)	$O(k \log^3 n)$
GCD	Euclidean	$O(\log \min(a,b))$
Modular inverse	Extended Euclidean	$O(\log m)$
$a^b \bmod m$	Binary exponentiation	$O(\log b)$
Factor n (small)	Trial division up to $\sqrt{n}$	$O(\sqrt{n})$

Recommended Next Topics

Finite Fields ($GF(2^m)$)

Extension for AES and elliptic curves

Elliptic Curve Cryptography

Modular arithmetic on elliptic curves

Chinese Remainder Theorem

Speeds up RSA decryption significantly

Post-Quantum Cryptography

Lattice-based (NTRU, Kyber) replacing prime-based schemes

📄 *Developed by Talenciaglobal* — This document is intended as a formal professional reference for engineers, cryptographers, and computer science practitioners working with number-theoretic algorithms.